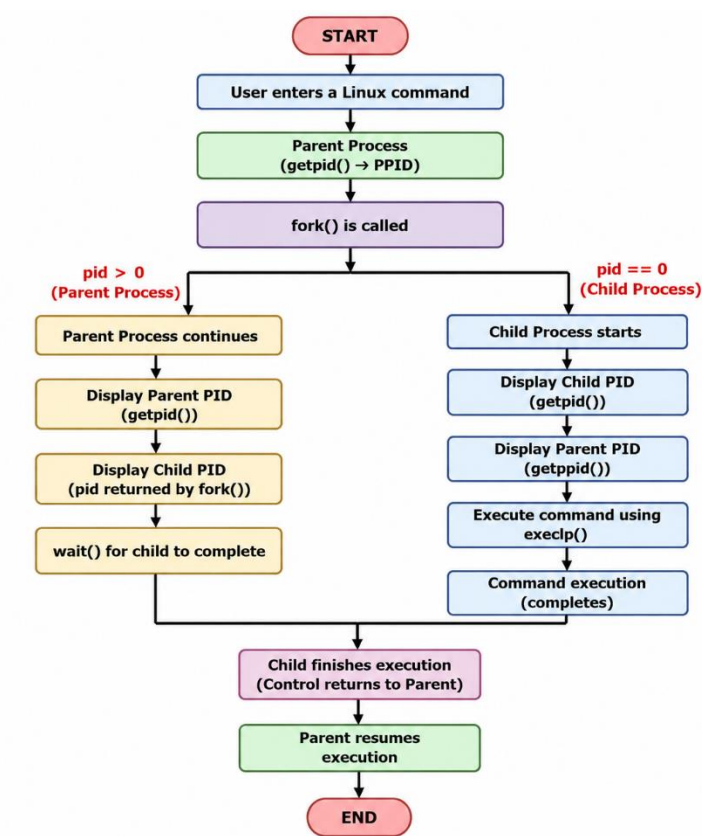


Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

- a) Write a C code to create a process using fork() system call.
- b) Read the user input as shell command.
- c) Use a system call exec() to execute the shell command.
- d) Use a wait() system call for waiting a parent process.
- e) Display the PIDs of parent and child processes.



Example Output:

Enter a Linux command: ls

Parent Process

Child Process

Parent PID : 67

Child PID : 68

Child PID : 68

Parent PID : 67

Executing command: ls

CE1 FIRST.c File2.c Proces_States.c command_executor filecopy.c
hello.c process_example session_4_a session_4_b.c

CE1.c File1.c First.c Process_States.c command_executor.c fork_example.c.save
my_shell process_example.c session_4_a.c

FIRST File1.txt Proces_States R1 filecopy hello
os-lab process_example.cyes session_4_b

Child process completed.

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char command[100];
```

```
    pid_t pid;
```

```
    printf("Enter a Linux command: ");
```

```
    fgets(command, sizeof(command), stdin);
```

```
    /* Remove newline from input */
```

```
command[strcspn(command, "\n")] = '\0';

pid = fork();

if (pid < 0)
{
    perror("fork failed");
    return 1;
}

else if (pid == 0)
{
    /* Child Process */
    printf("\nChild Process\n");
    printf("Child PID : %d\n", getpid());
    printf("Parent PID: %d\n", getppid());

    printf("\nExecuting command: %s\n\n", command);

    /* Execute command using shell */
    execl("/bin/sh", "sh", "-c", command, (char *)NULL);

    /* This executes only if exec fails */
    perror("exec failed");
    exit(1);
}

else
{
    /* Parent Process */
```

```
printf("\nParent Process\n");
printf("Parent PID: %d\n", getpid());
printf("Child PID : %d\n", pid);

printf("\nParent waiting for child...\n");

wait(NULL);

printf("\nChild process completed.\n");
printf("Parent process resumes execution.\n");
}

return 0;
}
```

OUTPUT:

chakrika@Chakrika:~/OSSP\$./Practical/Week-02/command_executor

Enter a Linux command: ls

Parent Process

Parent PID : 486

Child PID : 492

Child Process

Parent PID : 486

Child PID : 492

Executing command: ls

Practical Skill

Task-2: Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

CPU:

The OS abstracts the CPU by scheduling processes and allocating CPU time among them. Applications don't directly control the CPU; they request CPU resources through the operating system.

Memory:

The OS provides each process with its own virtual address space. Applications work with virtual memory rather than directly managing physical RAM.

Storage:

The OS provides files and directories as an abstraction over physical storage devices. Programs can use file operations without needing to know the physical location of data on the disk.

I/O

devices:

The OS uses device drivers and system calls to provide a common interface to hardware such as keyboards, displays, disks, and network devices.

Commands:

```
uname -a
lscpu
ps -ef
ps -eo pid,ppid,comm,%cpu,%mem,state
top
```

Result

1. uname -a

```
Linux Chakrika 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18
21:54:43 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
```

2. lscpu

```
Architecture:    x86_64
CPU(s):          16
Vendor ID:       GenuineIntel
Model name:      13th Gen Intel(R) Core(TM) i7-1360P
Thread(s) per core: 2
Core(s) per socket: 8
Socket(s):       1
Virtualization:  VT-x
Hypervisor vendor: Microsoft
Virtualization type: full
```

3. ps -ef

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	0	08:36	?	00:00:00	/sbin/init
root	2	1	0	08:36	hvc0	00:00:00	/init
root	55	1	0	08:36	?	00:00:00	/usr/lib/systemd/systemd-journald
root	102	1	0	08:36	?	00:00:00	/usr/lib/systemd/systemd-udevd
systemd+	111	1	0	08:36	?	00:00:00	/usr/lib/systemd/systemd-resolved
root	181	1	0	08:36	?	00:00:00	/usr/sbin/cron -f -P
root	184	1	0	08:36	?	00:00:00	@dbus-daemon --system
root	192	1	0	08:36	?	00:00:00	/usr/lib/systemd/systemd-logind
mysql	276	1	1	08:36	?	00:00:02	/usr/sbin/mysqld
chakrika	372	371	0	08:36	pts/0	00:00:00	-bash
chakrika	444	373	0	08:36	pts/1	00:00:00	-bash

4. ps -eo pid,ppid,comm,%cpu,%mem,state

PID	PPID	COMMAND	%CPU	%MEM	S
1	0	systemd	0.4	0.1	S
2	1	init-systemd(Ub	0.0	0.0	S
55	1	systemd-journal	0.0	0.2	S
102	1	systemd-udevd	0.0	0.0	S
111	1	systemd-resolve	0.0	0.1	S
118	1	systemd-timesyn	0.0	0.1	S
181	1	cron	0.0	0.0	S
184	1	dbus-daemon	0.0	0.0	S
276	1	mysqld	1.7	4.9	S
372	371	bash	0.0	0.0	S
444	373	bash	0.0	0.0	S
506	372	ps	0.0	0.0	R

5. top

Tasks: 23 total, 1 running, 22 sleeping, 0 stopped, 0 zombie

%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id

MiB Mem : 7783.7 total, 5566.1 free, 880.7 used, 1489.4 buff/cache

MiB Swap: 2048.0 total, 2048.0 free, 0.0 used

PID	USER	S	%CPU	%MEM	COMMAND
276	mysql	S	1.0	5.0	mysqld
507	chakrika	R	0.0	0.1	top